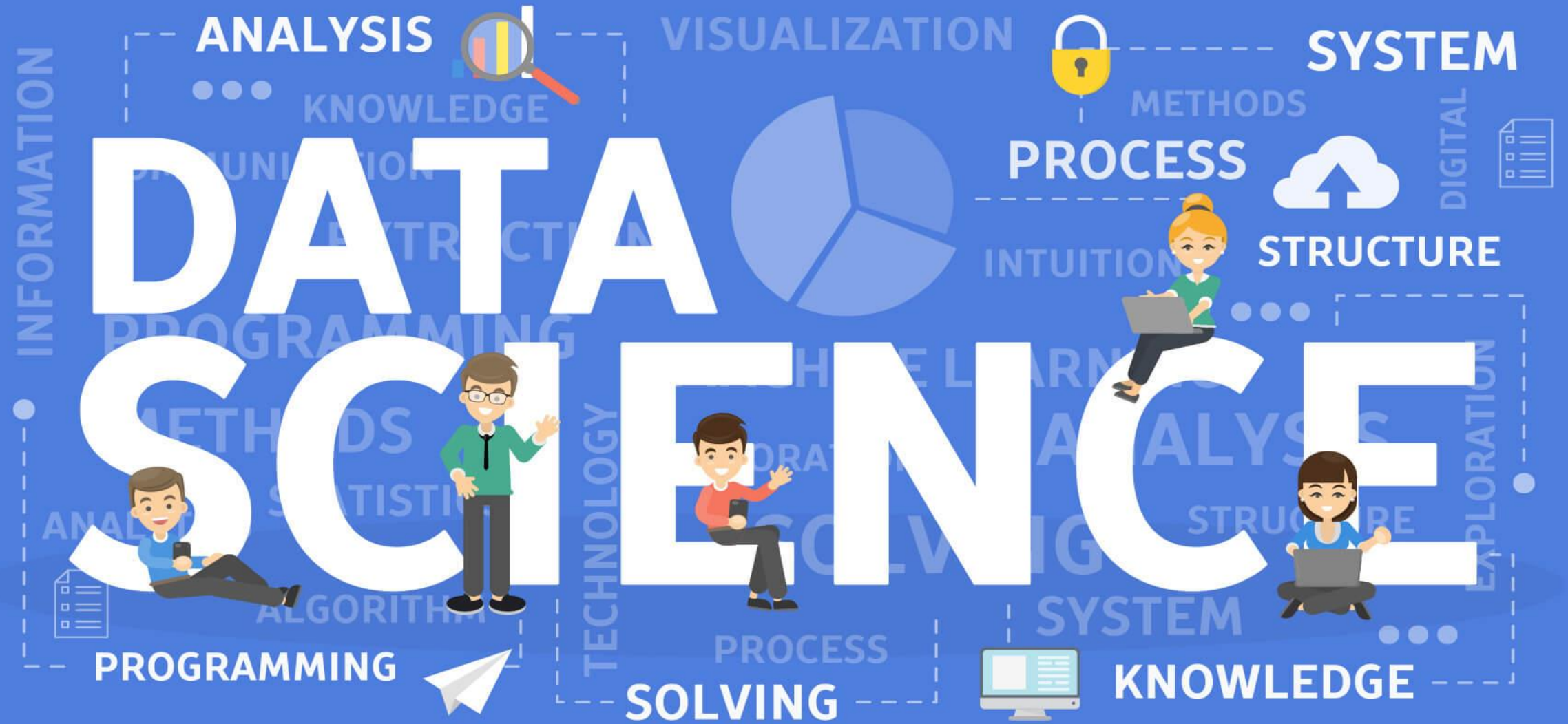




Ex:No.#05

Pima Indians Diabetes Dataset — Univariate, Bivariate and Multiple Regression Analysis

Dataset: the Pima Indians Diabetes Dataset

originally collected by the National Institute of Diabetes and Digestive and Kidney Diseases and distributed through the UCI Machine Learning Repository. It contains 768 female patients of Pima Indian heritage, 8 diagnostic measurements, and a binary target (Outcome: 1 = diabetic, 0 = not diabetic).

- (a) Univariate analysis — frequency, mean, median, mode, variance, standard deviation, skewness, kurtosis
- (b) Bivariate analysis — simple linear regression
- (c) Bivariate analysis — logistic regression
- (d) Multiple regression analysis (several predictors)

Univariate Analysis

Understanding a Single Variable



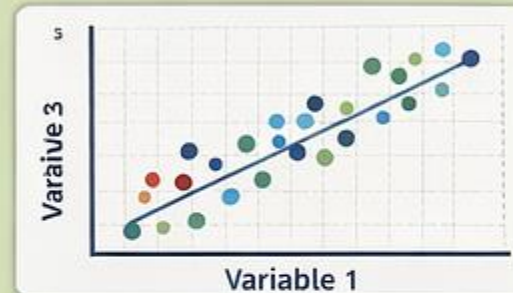
- Distribution
- Central Tendency
- Outliers



What does one variable look like?

Bivariate Analysis

Exploring Relationships Between Two Variables



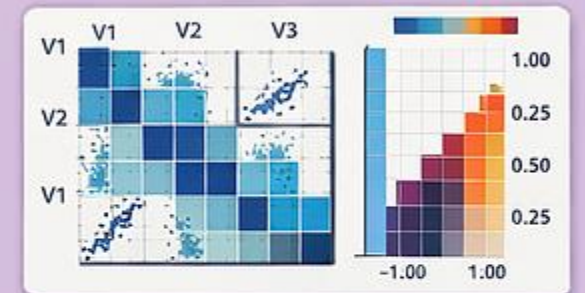
- Correlation
- Comparison
- Predictor Influence



How do two variables move together?

Multivariate Analysis

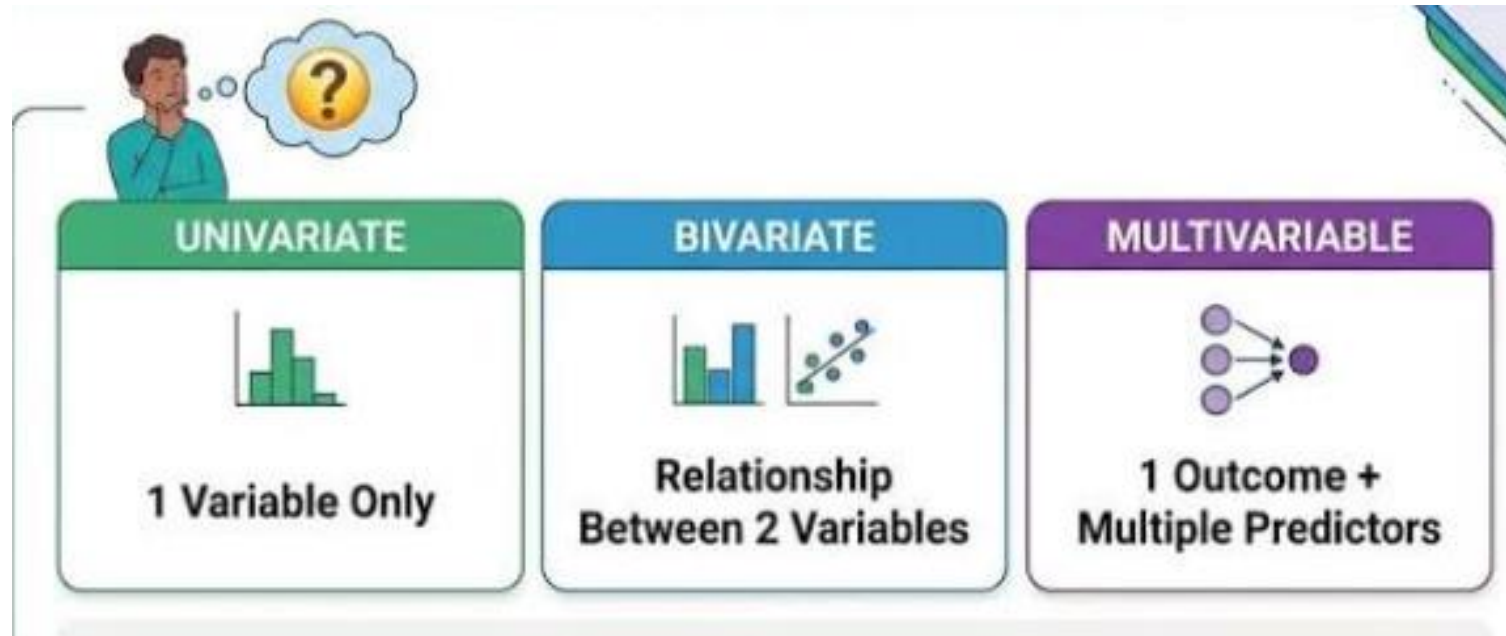
Seeing Multiple Variables Together



- Feature Interactions
- Correlation Heatmap
- Multicollinearity



How do many variables interact?



Univariate analysis is the simplest and easiest form of data analysis where the data being analyzed contains only one variable.

Example - Studying the heights of players in the NBA.

Univariate analysis can be described using **Central Tendency, Dispersion, Quartiles, Bar charts, Histograms, Pie charts, and Frequency distribution tables.**

The **bivariate analysis** involves the analysis of two variables to **find causes, relationships, and correlations between the variables.**

Example – Analyzing the sale of ice creams based on the temperature outside.

The bivariate analysis can be explained using **Correlation coefficients, Linear regression, Logistic regression, Scatter plots, and Box plots.**

Multivariate analysis involves the analysis of **three or more variables** to understand the relationship of each variable with the other variables.

Example – Analyzing Revenue based on expenditure.

Multivariate analysis can be performed using **Multiple regression, Factor analysis, Classification & regression trees, Cluster analysis, Principal component analysis, Dual-axis charts, etc.**

Univariate Statistics

1. Univariate analysis examines **one variable at a time** to understand its distribution and characteristics.

(a) Frequency

Frequency indicates **how many times each value or category occurs**.

Interpretation:

0 = Non-diabetic patients (500)

1 = Diabetic patients (268)

(b) mean

The **mean** is the arithmetic average of all observations.

Formula:

$$\text{Mean} = \frac{\sum X}{N}$$

Example:

If glucose values are

80, 90, 100

Mean

$= (80 + 90 + 100) / 3$

$= 90$

Purpose:

Represents the central value of the data.

- One Python script — every example prints real output
- Each section ends with a challenge; solutions live at the bottom
- Covers creation, filtering, missing data, groupby, merges, reshaping, apply/map, and time series
- Attached s

(c) Median

The **median** is the middle value after arranging the data in ascending order.

Example

80, 85, 90, 100, 120

Median = 90

Purpose:

- Less affected by extreme values (outliers).

(d) Mode

The **mode** is the value that appears most frequently.

Example

70, 80, 80, 90

Mode = 80

Purpose:

- Identifies the most common observation.

(e) Variance

Variance measures **how far the data points are spread from the mean**.

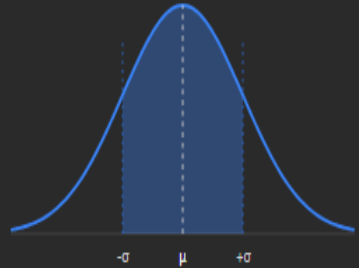
Purpose:

- Larger variance means the data are more dispersed.

$$\text{Var}(X) = E(X^2) - [E(X)]^2$$

$$\text{Var}(X) = (2.1)^2 = 4.41$$

$\text{Var}(X) \approx 4.41$



(f) Standard Deviation

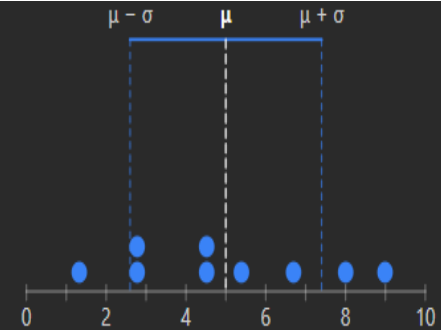
Standard deviation is the square root of the variance.

Purpose:

- Indicates how much the values deviate from the mean.
- Smaller standard deviation means data are clustered around the mean.

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$$

The points have mean $\mu = 5$ and $\sigma = 2.4$.



(g) Skewness

Skewness measures the **asymmetry** of a distribution.

- Positive skew → Right tail is longer.
- Negative skew → Left tail is longer.
- Zero skew → Symmetric distribution.

Interpretation:

- Glucose values in the diabetes dataset are often slightly **positively skewed**.

(h) Kurtosis

Kurtosis measures the **peakedness** or heaviness of the tails of a distribution.

- Positive kurtosis → Sharper peak and heavier tails.
- Negative kurtosis → Flatter distribution.
- Zero kurtosis → Similar to a normal distribution.

Purpose:

- Helps identify the presence of extreme values (outliers).

2. Bivariate Analysis – Linear Regression

Linear regression studies the relationship between **one independent variable (X)** and **one dependent variable (Y)**.

Example

Predict **Glucose** using **BMI**.

Model

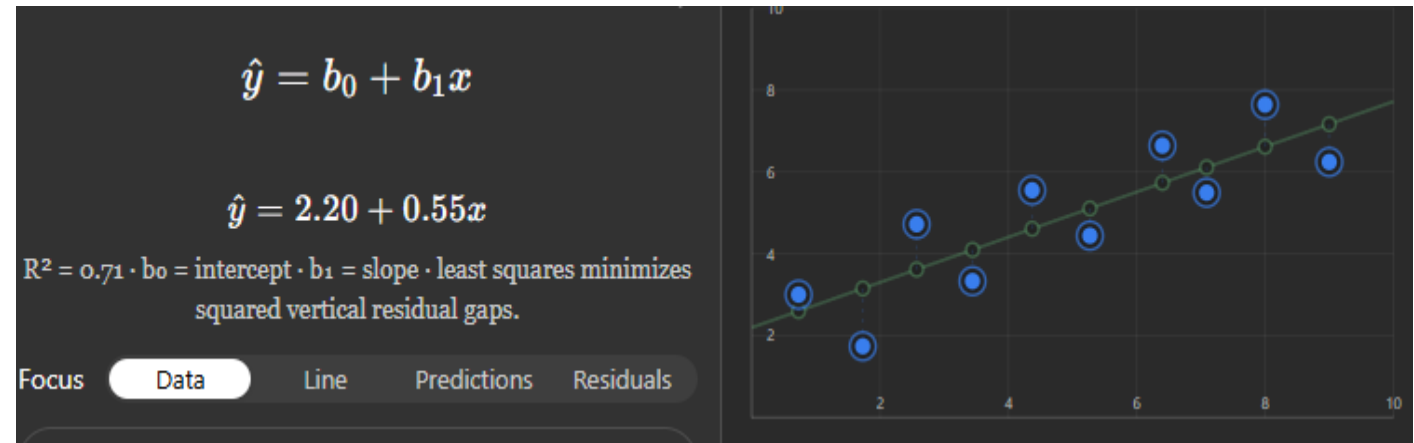
$$Y = b_0 + b_1X$$

Where

- **Y** = Glucose
- **X** = BMI
- **b₀** = Intercept
- **b₁** = Slope

Purpose:

- Predicts continuous numerical values.
- Measures the strength and direction of a linear relationship.



3. Bivariate Analysis – Logistic Regression

Logistic regression predicts a **binary (categorical) outcome**.

Example

Predict whether a patient has diabetes.

- Outcome = 0 → No Diabetes
- Outcome = 1 → Diabetes

Purpose:

- Used for classification rather than predicting continuous values.
- Produces the probability that an observation belongs to a class.

Typical evaluation metrics include:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score

4. Multiple Linear Regression

Multiple linear regression predicts one dependent variable using **two or more independent variables**.

Example

Predict **Glucose** using:

- Pregnancies
- Blood Pressure
- Skin Thickness
- Insulin
- BMI
- Age

Model

$$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n$$

Where:

- Y** = Dependent variable (Glucose)
- X₁, X₂, ...** = Independent variables
- b₀** = Intercept
- b₁, b₂, ...** = Regression coefficients

Purpose:

- Improves prediction by considering multiple factors simultaneously.
- Helps identify which variables have the greatest influence on the dependent variable.

5. Multiple Logistic Regression

Multiple Logistic Regression is a statistical and machine learning technique used to **predict a binary (two-class) outcome** using **two or more independent variables**.

In the **Pima Indians Diabetes Dataset**, the dependent variable is:

- **Outcome = 0** → Non-Diabetic
- **Outcome = 1** → Diabetic

The model uses several predictor variables such as:

- Pregnancies
- Glucose
- BloodPressure
- SkinThickness
- Insulin
- BMI
- DiabetesPedigreeFunction
- Age

to predict whether a patient has diabetes.

Mathematical Model

The multiple logistic regression model is:

$$\log \left(\frac{p}{1-p} \right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

Where:

- p = Probability that the patient has diabetes
- β_0 = Intercept
- $\beta_1, \beta_2, \dots$ = Coefficients
- $X_1, X_2, \dots$ = Predictor variables

Unlike linear regression, logistic regression predicts a **probability between 0 and 1**, which is then converted into a class (0 or 1).

Task	Method	Key Output
(a) Univariate	<code>.value_counts(), .mean(), .median(), .mode(), .var(), .std(), .skew(), .kurt()</code>	Distribution shape per variable
(b) Bivariate — linear	<code>sm.OLS(Insulin ~ Glucose)</code>	Slope, R^2, significance
(c) Bivariate — logistic	<code>sm.Logit(Outcome ~ Glucose)</code>	Odds ratio, accuracy, confusion matrix
(d) Multiple linear	<code>sm.OLS(BMI ~ Glucose+Age+SkinThickness+Insulin+BloodPressure)</code>	Adjusted R^2, coefficients, VIF
(d, extension)		
Multiple logistic	<code>sm.Logit(Outcome ~ all predictors)</code>	Odds ratios, ROC/AUCon

- One Python script — every example prints real output

- Each section ends with a challenge; solutions live at the bottom

- Covers creation, filtering, missing data, groupby, merges, reshaping, apply/map,

- Attached separately as `pandas_examples_and_challenges.py`

Column	Meaning
Pregnancies	Number of times pregnant
Glucose	Plasma glucose concentration (2-hr oral glucose tolerance test)
BloodPressure	Diastolic blood pressure (mm Hg)
SkinThickness	Triceps skin fold thickness (mm)
Insulin	2-hour serum insulin (mu U/ml)
BMI	Body mass index
DiabetesPedigreeFunction	Diabetes pedigree (genetic risk score)
Age	Age in years
Outcome	1 = diabetic, 0 = not diabetic

Steps	Description
(a) Univariate analysis	❖ for every variable: frequency (value counts / binned distribution), mean, median, mode, variance, std dev, skewness, kurtosis, plus histograms showing the shape of each distribution. ❖ treated biologically-impossible zeros (e.g. BMI = 0, BloodPressure = 0) as missing before computing stats, since those are data-entry placeholders, not real measurements — otherwise means/variances get distorted.

Steps	Description
Code	❖ <code>import pandas as pd</code> ❖ <code>import numpy as np</code> ❖ <code>import matplotlib.pyplot as plt</code> ❖ <code>import seaborn as sns</code> ❖ <code>import statsmodels.api as sm</code> ❖ <code>import statsmodels.formula.api as smf</code> ❖ <code>from statsmodels.stats.outliers_influence import variance_inflation_factor</code> ❖ <code>from sklearn.linear_model import LogisticRegression</code> ❖ <code>from sklearn.model_selection import train_test_split</code> ❖ <code>from sklearn.metrics import confusion_matrix, classification_report, accuracy_score, roc_curve, roc_auc_score</code> ❖ <code>pd.set_option('display.max_columns', None)</code> ❖ <code>plt.rcParams['figure.figsize'] = (7, 4)</code>

Steps	Description
Load the Data	<pre>url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv' cols = ['Pregnancies','Glucose','BloodPressure','SkinThickness','Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age','Outcome'] try: df = pd.read_csv(url, names=cols) except Exception: df = pd.read_csv('pima_diabetes.csv') print(df.shape) df.head() df.info() zero_as_missing = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI'] df_clean = df.copy() df_clean[zero_as_missing] = df_clean[zero_as_missing].replace(0, np.nan) df_clean.isnull().sum()</pre> • One Python script — every example prints real output • Each section ends with a challenge; solutions live at the bottom • Covers creation, filtering, missing data, groupby, merges, reshaping, apply/map, vectorizing, etc. • Attached separately as pandas_examples_and_challenges.py

Steps	Description
(a) Univariate Analysis: For each numeric variable: frequency (value counts / distribution),	Frequency — categorical / discrete variables <pre>df['Outcome'].value_counts() df['Outcome'].value_counts(normalize=True) * 100 # class balance, in % df['Pregnancies'].value_counts().sort_index() # discrete count variable</pre> Frequency distribution — continuous variables (binned) <pre>pd.cut(df_clean['Glucose'], bins=6).value_counts().sort_index()</pre> • Attached separately as pandas_examples_and_challenges.py <pre>df_clean.hist(figsize=(11, 8), bins=20) plt.suptitle('Frequency distribution of each variable') plt.tight_layout() plt.show()</pre>

Steps	Description
(a) Univariate Analysis: mean, median, mode, variance, standard deviation,	Mean, median, mode, variance, standard deviation <pre> numeric_cols = [c for c in df_clean.columns if c != 'Outcome'] summary = pd.DataFrame({ 'mean': df_clean[numeric_cols].mean(), 'median': df_clean[numeric_cols].median(), 'mode': df_clean[numeric_cols].mode().iloc[0], 'variance': df_clean[numeric_cols].var(), 'std_dev': df_clean[numeric_cols].std(), }) summary </pre>

- Attached separately as pandas_examples_and_challenges.py

Steps	Description
(a) Univariate Analysis:	Skewness $> 0 \rightarrow$ right-tailed distribution; $< 0 \rightarrow$ left-tailed; $\sim 0 \rightarrow$ roughly symmetric. Kurtosis (excess) $> 0 \rightarrow$ heavier tails/more outliers than a normal distribution; $< 0 \rightarrow$ lighter tails.
skewness, and kurtosis.	<pre>shape_stats = pd.DataFrame({ 'skewness': df_clean[numeric_cols].skew(), 'kurtosis': df_clean[numeric_cols].kurt(),})</pre> shape_stats
	<pre>fig, axes = plt.subplots(2, 4, figsize=(15, 6)) for ax, col in zip(axes.flat, numeric_cols): sns.histplot(df_clean[col].dropna(), kde=True, ax=ax) ax.set_title(f'{col}\nskew={df_clean[col].skew():.2f},kurt={df_clean[col].kurt():.2f}') plt.tight_layout() plt.show()</pre>
	<pre>df_clean.describe().T</pre> # one-shot summary: count, mean, std, min, quartiles, max

Steps	Description
(b) Bivariate Analysis — Linear Regression	Question: does plasma glucose concentration predict serum insulin level? Both are continuous, and physiologically linked (insulin resistance raises glucose), which makes them a natural pair for simple linear regression. Model: $\text{Insulin} = \beta_0 + \beta_1 * \text{Glucose} + \epsilon$
	<pre>lr_data = df_clean[['Glucose', 'Insulin']].dropna() X = sm.add_constant(lr_data['Glucose']) y = lr_data['Insulin'] linear_model = sm.OLS(y, X).fit() print(linear_model.summary())</pre>
	<pre>plt.scatter(lr_data['Glucose'], lr_data['Insulin'], alpha=0.4, label='Observed') x_range = np.linspace(lr_data['Glucose'].min(), lr_data['Glucose'].max(), 100) y_pred = linear_model.params['const'] + linear_model.params['Glucose'] * x_range plt.plot(x_range, y_pred, color='red', label='Fitted line') plt.xlabel('Glucose') plt.ylabel('Insulin') plt.title(f"Linear Regression: Insulin ~ Glucose (R² = {linear_model.rsquared:.3f})") plt.legend() plt.show()</pre>

Steps	Description
(b) Bivariate Analysis — Logistic Regression	Question: does glucose level predict whether a patient is diabetic (Outcome)? Since Outcome is binary, logistic regression (not linear regression) is the correct model. Model: $\text{logit}(P(\text{Outcome}=1)) = \beta_0 + \beta_1 * \text{Glucose}$
	<pre>log_data = df_clean[['Glucose', 'Outcome']].dropna() X_log = sm.add_constant(log_data['Glucose']) y_log = log_data['Outcome'] logit_model = sm.Logit(y_log, X_log).fit() print(logit_model.summary())</pre> • Attached separately as pandas_examples_and_challenges.py <pre>odds_ratio = np.exp(logit_model.params['Glucose']) print(f'Odds ratio for Glucose: {odds_ratio:.4f}') print(f'Interpretation: each 1-unit increase in glucose multiplies the odds of " f"diabetes by {odds_ratio:.4f} (i.e. a {(odds_ratio-1)*100:.2f}% increase in odds).")</pre>

Steps	Description
(b) Bivariate Analysis — Logistic Regression	<pre> # Sigmoid curve: predicted probability of diabetes across the glucose range x_range = np.linspace(log_data['Glucose'].min(), log_data['Glucose'].max(), 200) X_range = sm.add_constant(x_range) probs = logit_model.predict(X_range) plt.scatter(log_data['Glucose'], log_data['Outcome'], alpha=0.3, label='Observed (0/1)') plt.plot(x_range, probs, color='red', linewidth=2, label='Predicted P(Outcome=1)') plt.xlabel('Glucose') plt.ylabel('P(Diabetes)') plt.title('Logistic Regression: Outcome ~ Glucose') plt.legend() plt.show() </pre>

Steps	Description
(b) Bivariate Analysis — Logistic Regression	<pre># Classification performance using a 0.5 probability cutoff pred_labels = (logit_model.predict(X_log) >= 0.5).astype(int) cm = confusion_matrix(y_log, pred_labels) print("Confusion matrix:\n", cm) print("\nAccuracy:", accuracy_score(y_log, pred_labels)) print("\n", classification_report(y_log, pred_labels))</pre>
	<pre>sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Predicted 0', 'Predicted 1'], yticklabels=['Actual 0', 'Actual 1']) plt.title('Confusion Matrix — Logistic Regression (Glucose only)') plt.show()</pre>

Steps	Description
(c) Multiple Regression Analysis	Simple regression uses one predictor; multiple regression extends the same idea to several predictors at once, so each coefficient reflects that variable's effect holding the others constant.
Multiple linear regression	Question: can BMI be predicted from a combination of Glucose, Age, SkinThickness, Insulin, and BloodPressure? Model: $BMI = \beta_0 + \beta_1 \cdot \text{Glucose} + \beta_2 \cdot \text{Age} + \beta_3 \cdot \text{SkinThickness} + \beta_4 \cdot \text{Insulin} + \beta_5 \cdot \text{BloodPressure} + \epsilon$ - Covers creation, filtering, missing data, groupby, merges, reshaping, apply/map, and time series - Attached separately as pandas_examples_and_challenges.py

Steps	Description
(c) Multiple linear regression	<pre> mreg_cols = ['Glucose', 'Age', 'SkinThickness', 'Insulin', 'BloodPressure', 'BMI'] mreg_data = df_clean[mreg_cols].dropna() X_multi = sm.add_constant(mreg_data.drop(columns='BMI')) y_multi = mreg_data['BMI'] multi_linear_model = sm.OLS(y_multi, X_multi).fit() print(multi_linear_model.summary()) </pre>
	<pre> fitted_vals = multi_linear_model.fittedvalues plt.scatter(y_multi, fitted_vals, alpha=0.4) plt.plot([y_multi.min(), y_multi.max()], [y_multi.min(), y_multi.max()], color='red') plt.xlabel('Actual BMI') plt.ylabel('Predicted BMI') plt.title(f"Multiple Linear Regression: Actual vs Predicted BMI (R² = {multi_linear_model.rsquared:.3f})") plt.show() </pre>

Steps	Description
(d) Multiple logistic Regression	The same multi-predictor idea applied to the binary outcome: predicting Outcome from all available features together, rather than Glucose alone as in part (c).
	<pre> feature_cols = ['Pregnancies','Glucose','BloodPressure','SkinThickness', 'Insulin','BMI','DiabetesPedigreeFunction','Age'] multi_log_data = df_clean[feature_cols + ['Outcome']].dropna() X_multi_log = sm.add_constant(multi_log_data[feature_cols]) y_multi_log = multi_log_data['Outcome'] multi_logit_model = sm.Logit(y_multi_log, X_multi_log).fit() print(multi_logit_model.summary()) </pre>

Steps	Description
(d) Multiple logistic Regression	The same multi-predictor idea applied to the binary outcome: predicting Outcome from all available features together, rather than Glucose alone as in part (c).
	<pre>odds_ratios = np.exp(multi_logit_model.params).drop('const').sort_values(ascending=False) print("Odds ratios (each predictor's effect, holding the others constant):") odds_ratios</pre> • One Python script — every example prints real output • Each section ends with a challenge; solutions live at the bottom • Covers creation, filtering, missing data, groupby, merges, reshaping, apply/map, and time series • Attached separately as pandas_examples_and_challenges.py

Steps	Description
Multiple logistic regression	<pre> X_train, X_test, y_train, y_test = train_test_split(multi_log_data[feature_cols], multi_log_data['Outcome'], test_size=0.25, random_state=42, stratify=multi_log_data['Outcome']) sk_model = LogisticRegression(max_iter=1000) sk_model.fit(X_train, y_train) y_pred = sk_model.predict(X_test) print("Test accuracy:", accuracy_score(y_test, y_pred)) print("\n", classification_report(y_test, y_pred)) fpr, tpr, _ = roc_curve(y_test, sk_model.predict_proba(X_test)[:, 1]) auc = roc_auc_score(y_test, sk_model.predict_proba(X_test)[:, 1]) plt.plot(fpr, tpr, label=f'AUC = {auc:.3f}') plt.plot([0, 1], [0, 1], linestyle='--', color='gray') plt.xlabel('False Positive Rate') plt.ylabel('True Positive Rate') plt.title('ROC Curve — Multiple Logistic Regression (all predictors)')plt.legend()plt.show() </pre>

Univariate	Bivariate	Multivariate
It only summarises a single variable at a time.	It only summarise two variables.	It only summarises more than 2 variables.
It does not deal with causes and relationships.	It does deal with causes and relationships and analysis is done.	It does not deal with causes and relationships and analysis is done.
It does not contain any dependent variable.	It does contain only one dependent variable.	It is similar to bivariate but it contains more than 2 variables.
The main purpose is to describe.	The main purpose is to explain.	The main purpose is to study the relationship among them.